

A miniature DynamoDB with replication

DynaLITE

Project under Prof. Kishore Kothapalli

DS Project by :

Kushagra Trivedi (2021101077)

Ronit Jalihal (2023101028)

Prasoon Dev(2021101090)

Date: 29th November 2025

Motivation and Architecture

- **The "Always-On" Requirement:** Modern services like Shopping Carts require high write availability; they cannot reject items even during network partitions.
- **The CAP Trade-off:** DynaLite explicitly chooses Availability (A) and Partition Tolerance (P) over strict Consistency, accepting Eventual Consistency.
- **Implementation:** Built using Python, RPyC (for RPC communication), and Redis (for persistent storage).

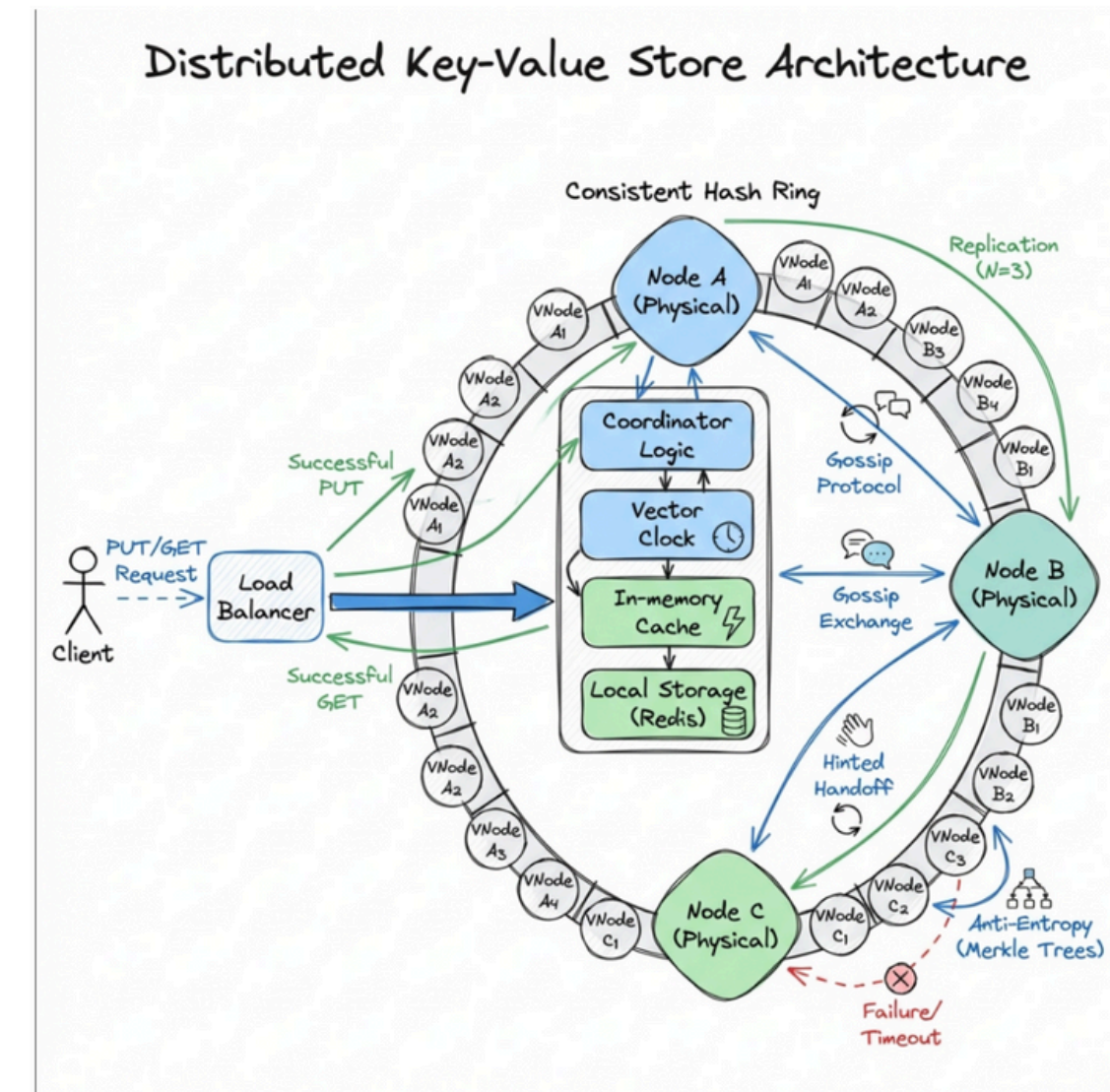


Fig. 1. DynamoMINI redundancy and replication architecture showing how data is replicated across multiple nodes for fault tolerance and high availability.

Data Partitioning: Consistent Hashing

- The Ring: Uses MD5 hashing to map both data and nodes to a 128-bit circular hash space¹⁰.
- Virtual Nodes: Each physical node claims multiple positions to ensure load balancing¹¹.
- Preference List: Data is replicated to the first N distinct physical nodes clockwise from the key¹².

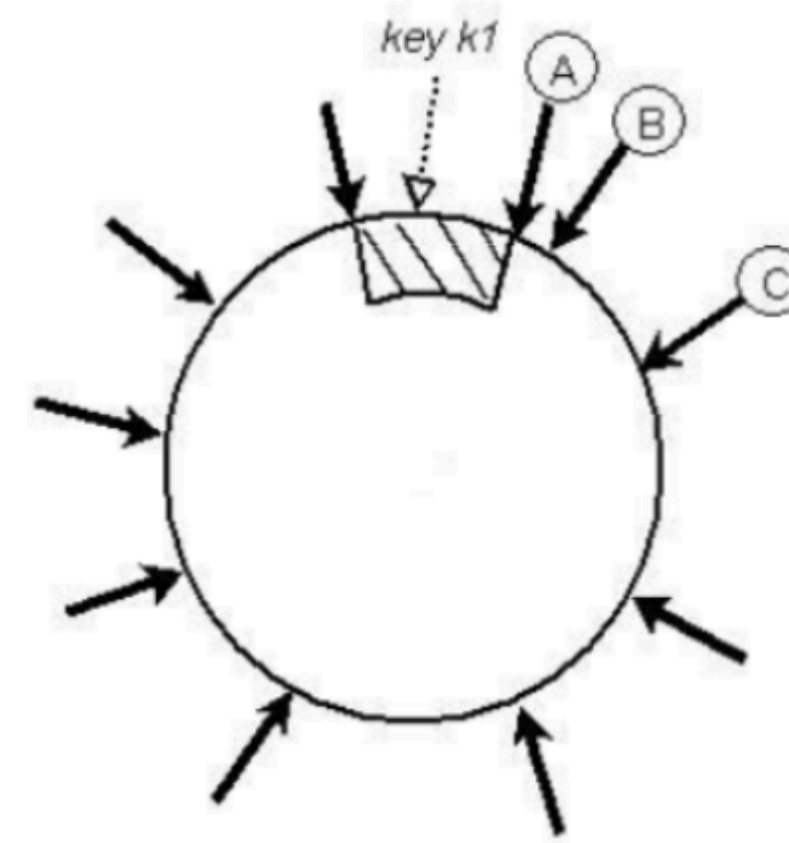


Fig. 7. Data partitioning and replication on the hash ring. Key K's preference list consists of nodes A, B, C (N=3), which store replicas of all keys in the shaded range.

Vector Clocks: Tracking Causality

- **Why Not Timestamps?** Physical timestamps are unreliable in distributed systems due to clock skew.
- **The Solution:** We use Vector Clocks [NodeID: Counter] to track the "happens-before" relationship.
- **Conflict Detection:**
 - If $VC1 < VC2$, it is an update.
 - If neither dominates, it is a Concurrent Write (Conflict).
- **Resolution:** Conflicts are returned to the client (or resolved during reads) for reconciliation.

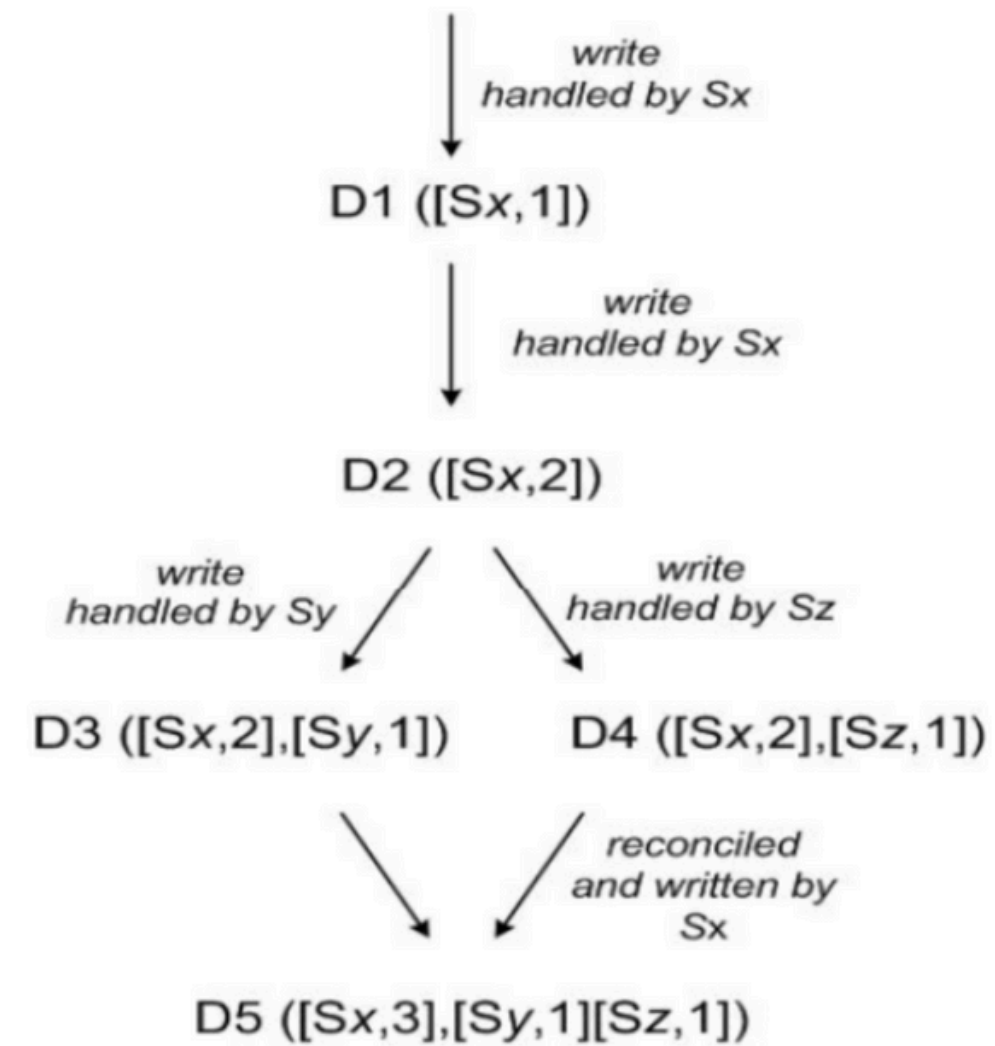


Fig. 4. Version evolution of an object showing vector clock progression. Concurrent writes create version branches that must be reconciled during reads.

Quorum-Based Replication

- **Tunable Consistency:** We use three parameters to tune the system:
 - N: Total Replicas (Replication Factor).
 - W: Write Quorum (Minimum acknowledgments for success).
 - R: Read Quorum (Minimum responses for a read).
- **The Guarantee:** We enforce $R + W > N$ to ensure at least one node in the read quorum has the latest write.
- **Typical Config:** N=3, R=2, W=2 (Balanced approach).

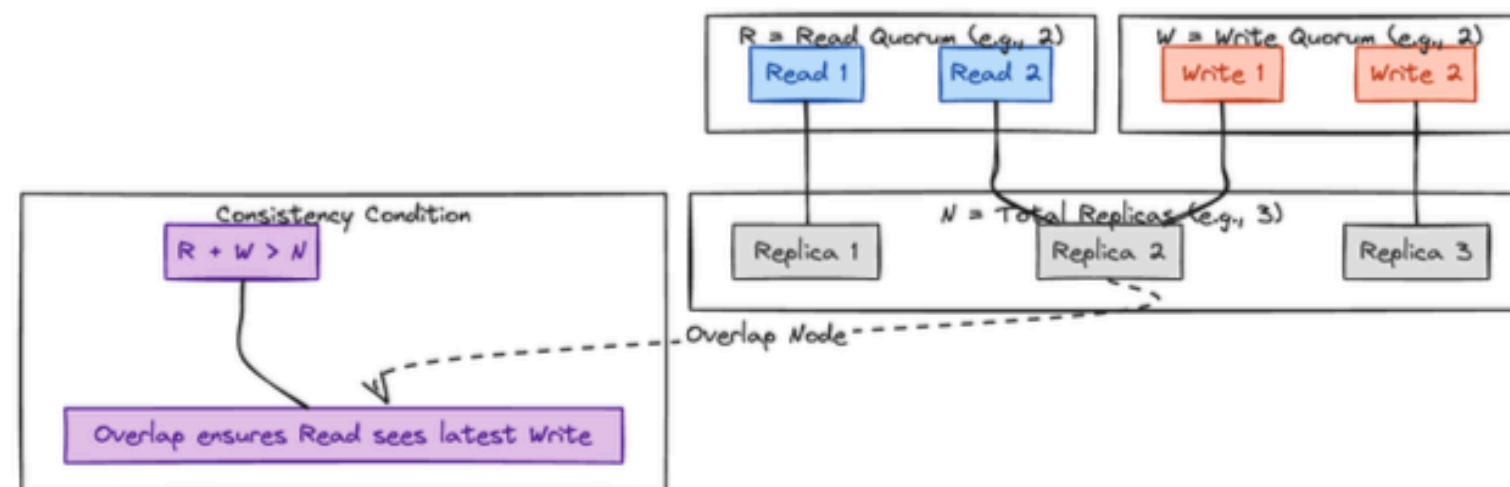


Fig. 5. Quorum technique with N=3, R=2, W=2. The overlap between read and write quorums guarantees consistency.

The PUT Operation (Write Path)

- **Step 1:** Coordinator generates a new Vector Clock and writes locally.
- **Step 2 (Synchronous):** Sends replication requests to N-1 nodes. Waits for W-1 acknowledgments.
- **Step 3 (Asynchronous):** Once the quorum is met, success is returned to the client immediately.
- **Step 4:** Remaining replicas are updated in the background to reduce latency.

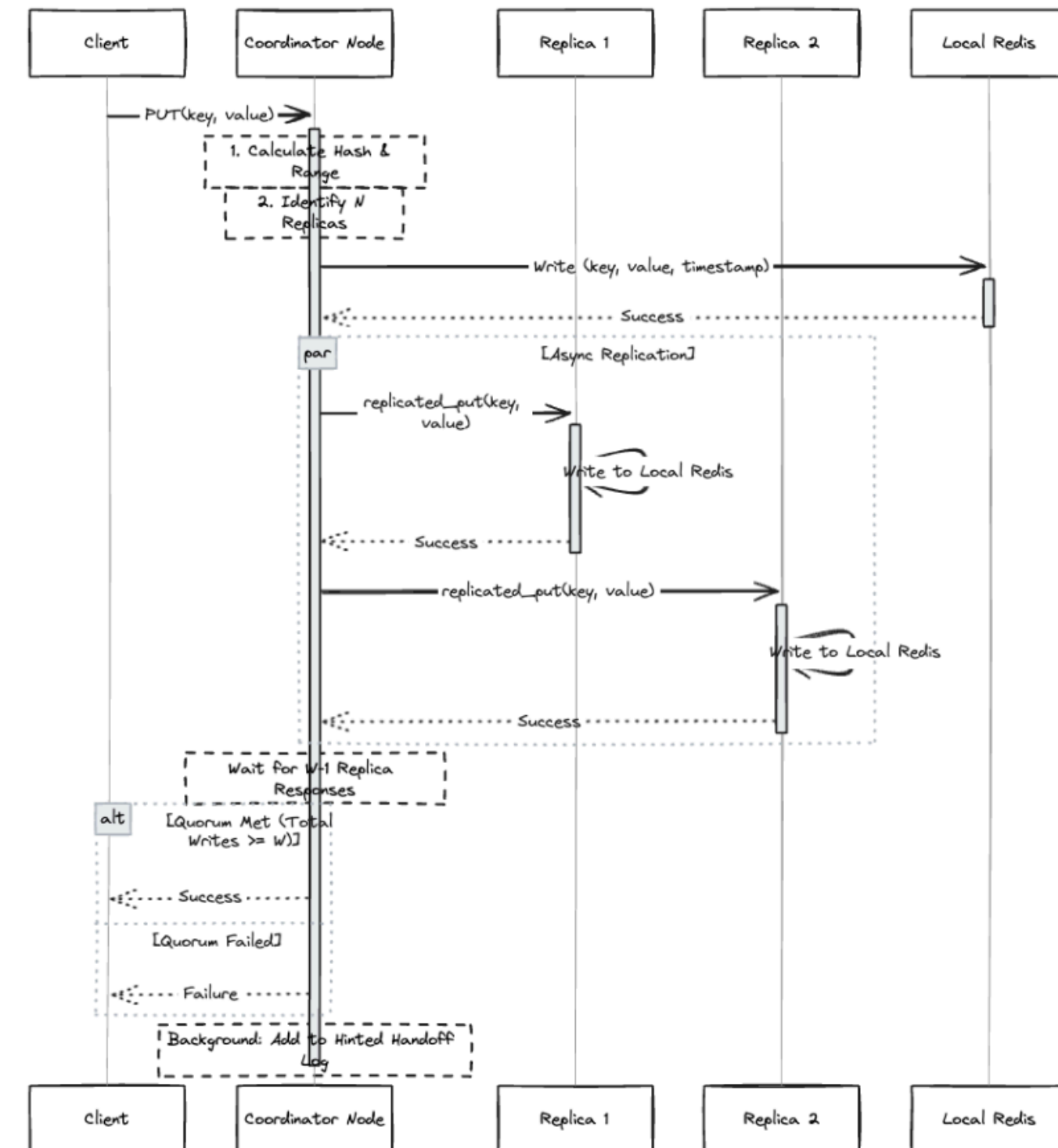


Fig. 11. PUT operation sequence showing quorum-based replication with $W=2$. Coordinator writes locally and waits for one additional replica acknowledgment before responding.

The GET Operation (Read Path)

Step 1: Client queries any node; request is forwarded to the Coordinator16.

Step 2: Coordinator requests data from N-1 replicas and waits for R responses.

Step 3 (Reconciliation): Vector Clocks are compared. If conflicts exist, all concurrent versions are returned18.

Step 4 (Read Repair): If a replica returns stale data, the coordinator asynchronously updates it with the fresh version19.

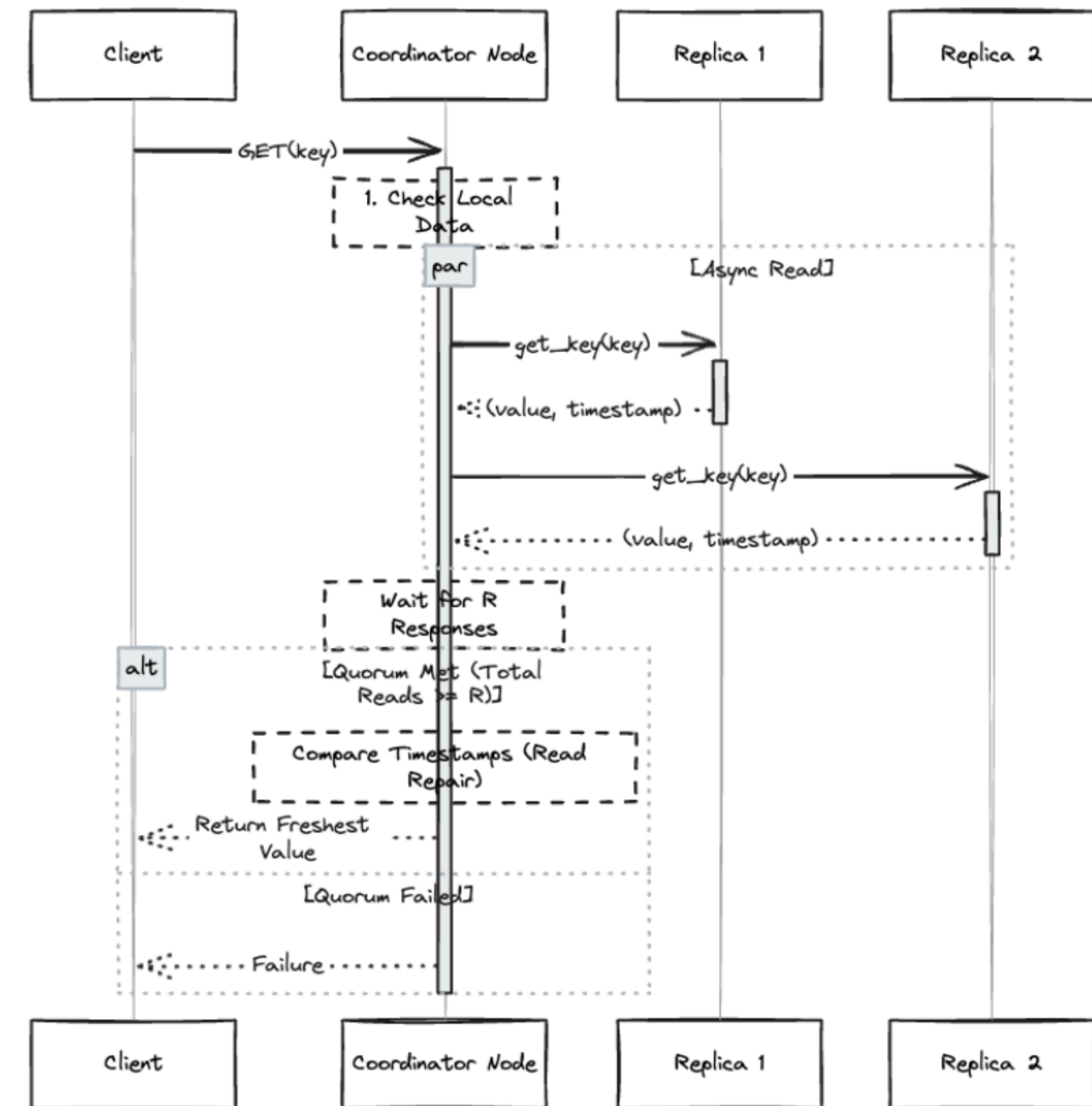


Fig. 12. GET operation sequence showing quorum-based read with R=2 and read repair mechanism for consistency maintenance.

Fault Tolerance: Hinted Handoff

- **Problem:** What if a replica is temporarily down during a write?
- **Solution:** We do not fail the write. The coordinator stores a "Hint" locally.
- **Recovery:** A background thread periodically scans for the node's recovery.
- **Replay:** Once the node is back online, the stored hints are replayed to ensure no data is lost.

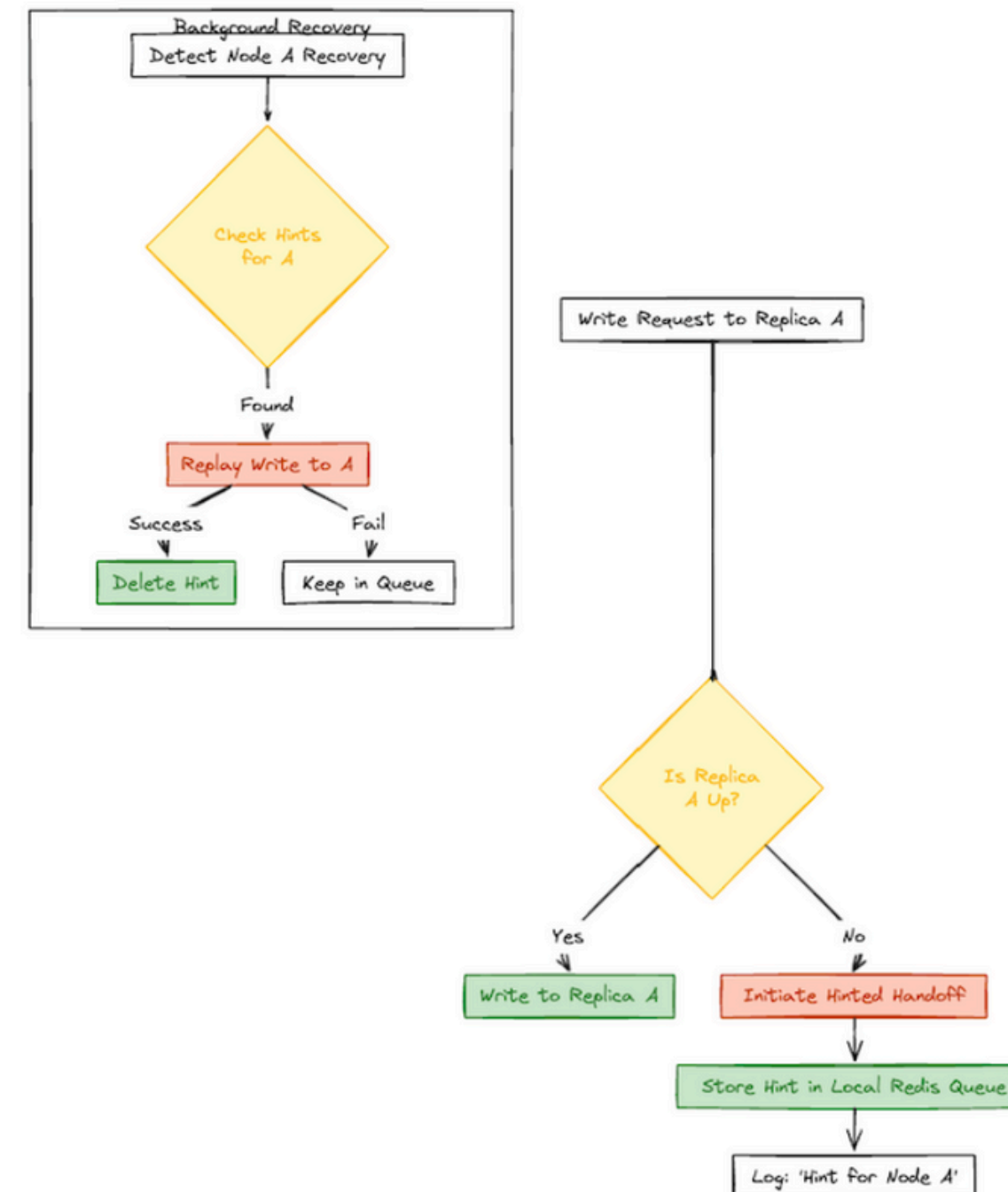


Fig. 13. Hinted handoff mechanism for handling temporary node failures and ensuring eventual consistency.

Background Synchronization: Merkle Trees

- **The Challenge:** Syncing replicas without sending the entire database over the network.
- **The Solution:** We use Merkle Trees (Hash Trees).
- **Root Hash:** Summarizes the entire key range.
- **Efficiency:** Nodes compare Root Hashes. If they match, data is consistent. If not, they descend the tree to find only the specific keys that differ.

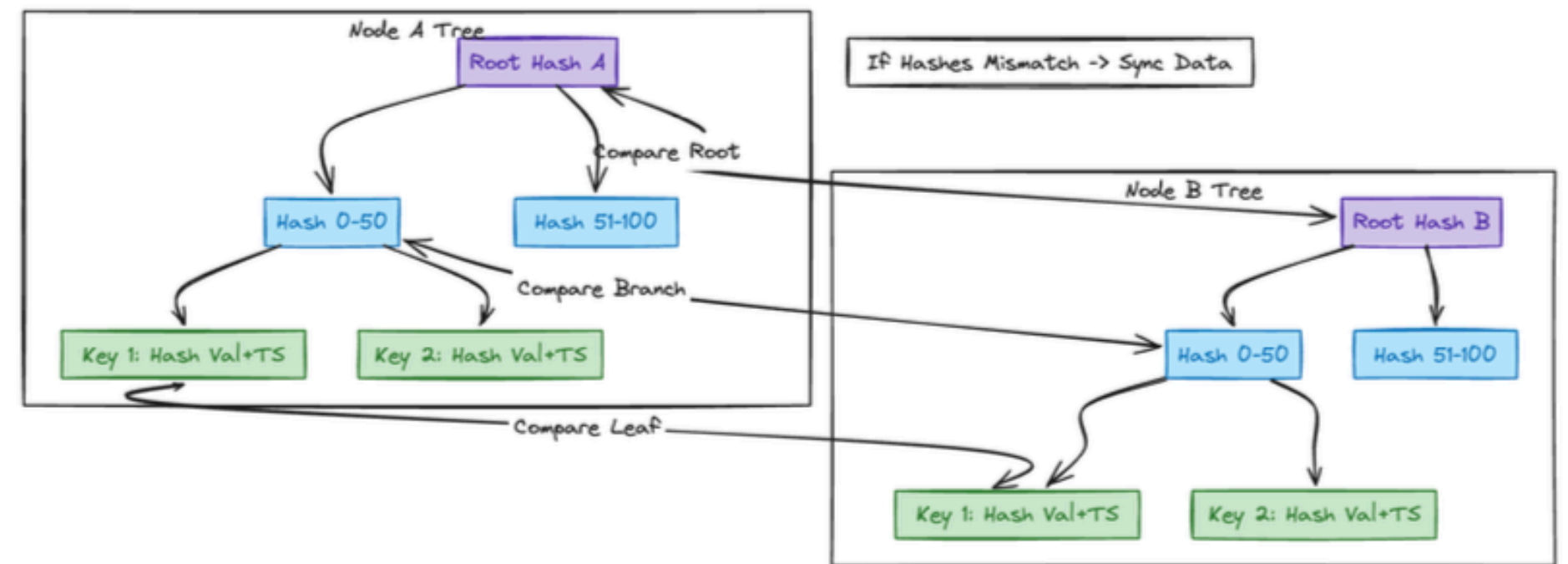
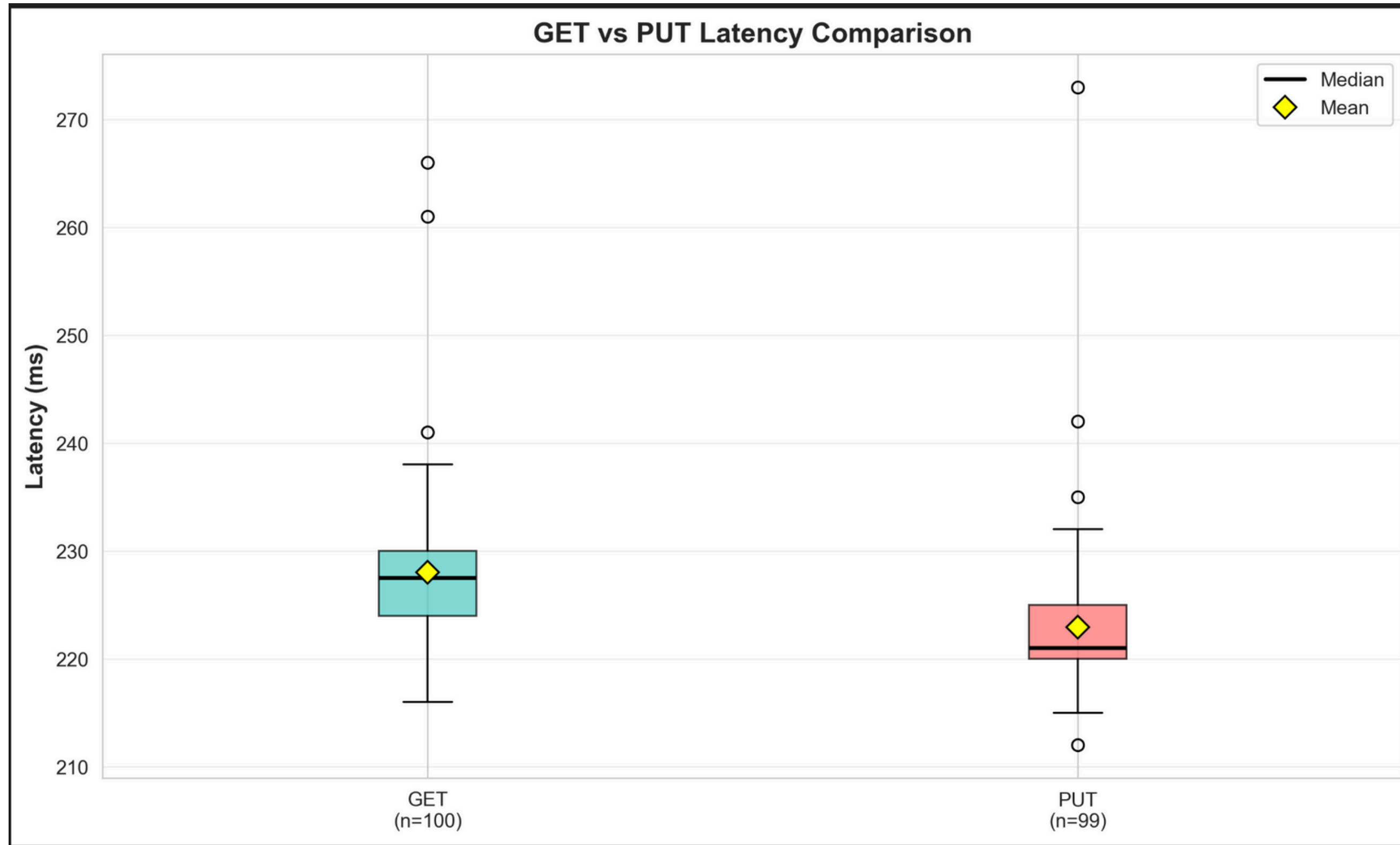


Fig. 14. Merkle tree-based anti-entropy for efficient replica synchronization. Only divergent branches require data exchange.

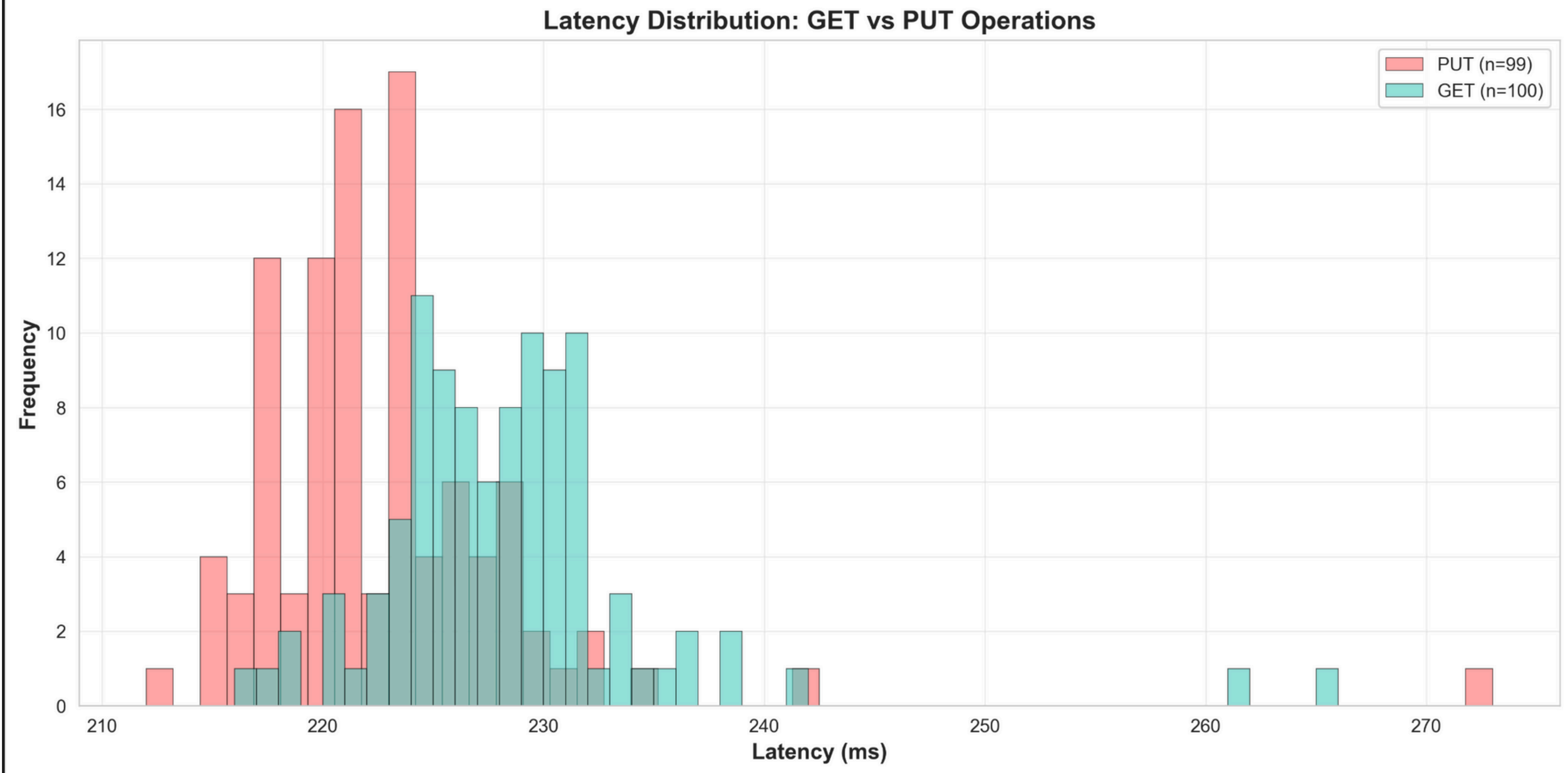
Demo

We have used Python as our programming language of choice, Rpyc for communication between processes and servers, and Redis to store the backend key-value data required for each node persistently.

Performance Analysis: Latency Graphs



Performance Analysis: Latency Graphs



Complete Latency Statistics

Put Latency Stats

Sample Size: 100

Average: 222.92 ms

Median (p50): 221.00 ms

p95: 231.10 ms

p99: 242.62 ms

Min: 212.00 ms

Max: 273.00 ms

Std Dev: 6.92 ms

Get Latency Stats

Sample Size: 100

Average: 228.00 ms

Median (p50): 227.50 ms

p95: 236.10 ms

p99: 261.05 ms

Min: 216.00 ms

Max: 266.00 ms

Std Dev: 6.76 ms

Throughput Stats

Total Operations: 199

Successful: 199 (100.0%)

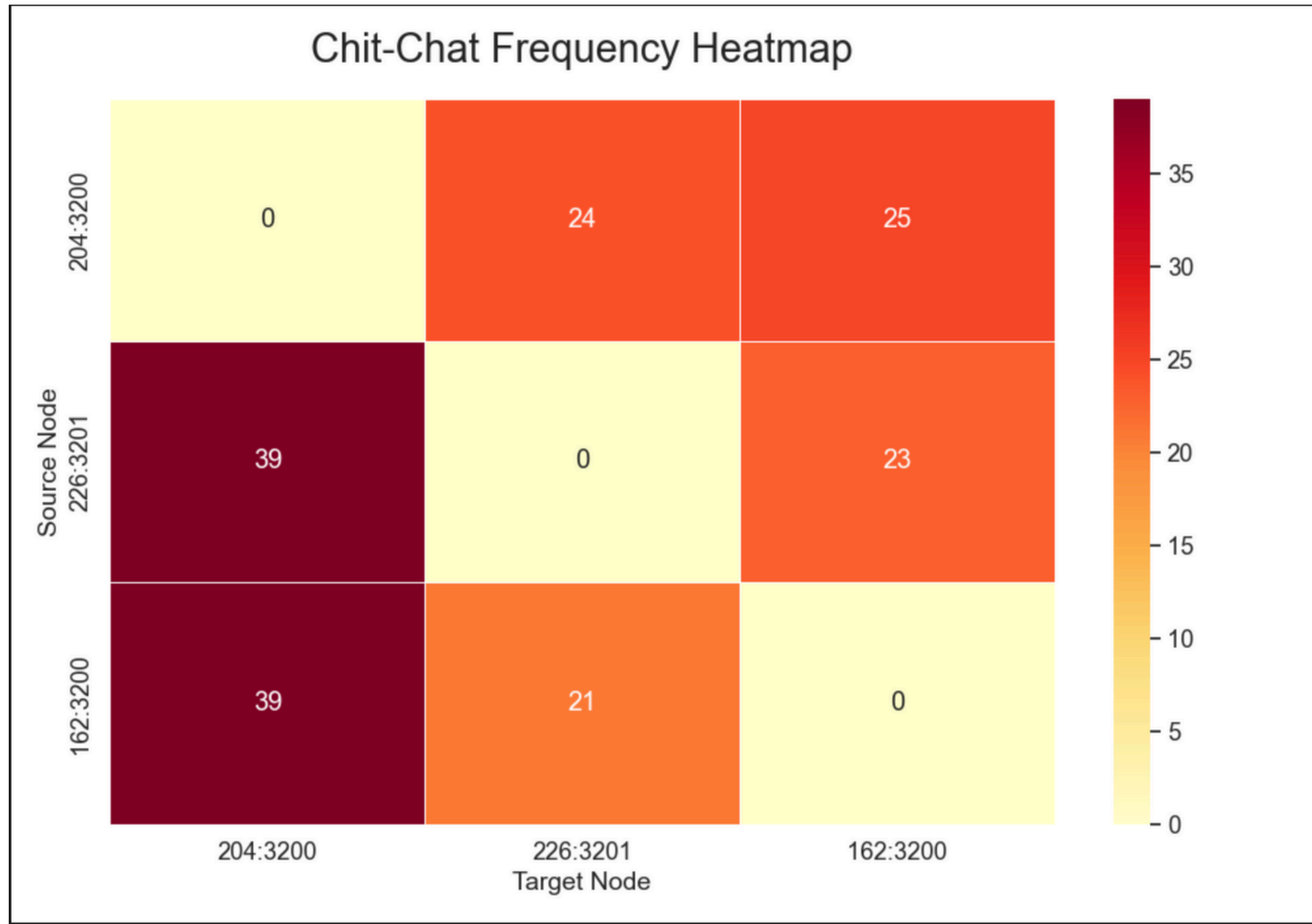
Time Span: 63.00 seconds

Throughput: 3.16 ops/sec

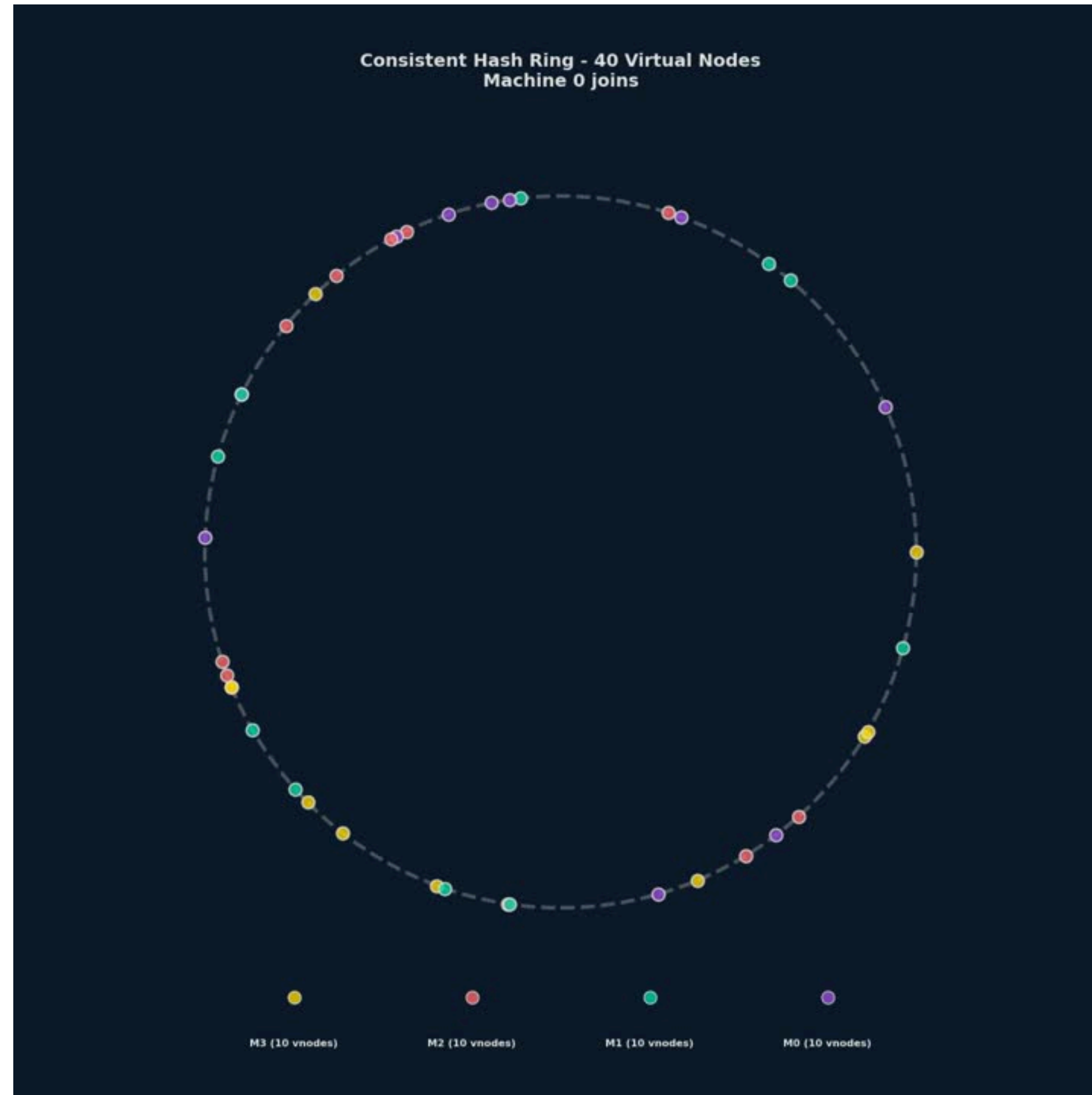
PUT Throughput: 1.57 ops/sec

GET Throughput: 1.59 ops/sec

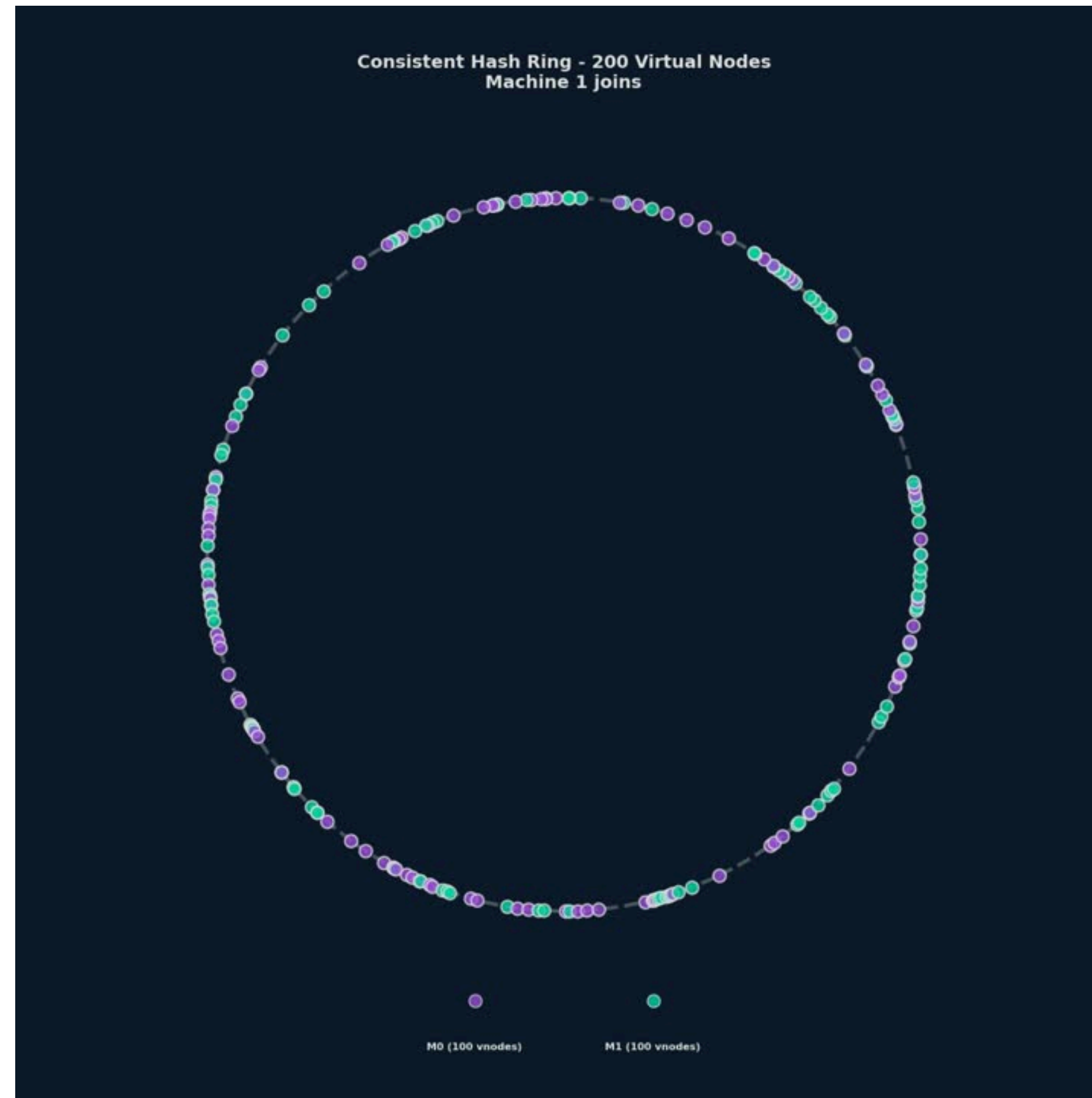
Gossiping Heatmap Analysis



Consistent Hashing Demonstration: small scale



Consistent Hashing Demonstration: large scale





DynamoLite

Thank You

30th November, 2025